

ZYLO — Backend + Frontend Integration PRD

Production-oriented Live Streaming Social Platform MVP

Product: Zylo — Live Bolder

Document: Backend Architecture, Media, Realtime, Storage, Scheduling & Frontend Integration PRD

Status: Implementation-ready

Primary stack: Next.js + Node.js/Express + TypeScript + PostgreSQL/Neon + Prisma + Redis + Socket.IO

Media: MediaMTX + FFmpeg for the free MVP; Cloudflare R2 for recorded streams; Cloudinary for user/content images

Docker: Explicitly excluded for this implementation phase

1. Purpose

This PRD extends the original live-streaming assessment into a production-oriented MVP architecture while keeping the implementation modular and deployable with managed services.

The backend must support:

- Authentication and authorization
- `NORMAL\_USER`, `CREATOR`, and `ADMIN` roles
- Profiles and follow/unfollow
- Immediate live streaming
- Future stream scheduling
- Live viewer presence and accurate-enough viewer counts
- Real-time stream chat
- Virtual gifts and wallet credits
- Stream recording
- Cloud storage and VOD playback
- Live/VOD quality selection
- VOD playback-speed selection
- Redis caching and presence
- Rate limiting
- Cloudinary media uploads and optimized delivery
- Admin moderation
- Reports
- Swagger/OpenAPI
- Tests
- Frontend API and Socket.IO integration
- Security and ownership enforcement

The original assessment remains the functional baseline. This document adds the requested scheduling, recording, storage, media optimization, caching, presence and frontend-integration requirements.

2. Important Scope Decisions

2.1 User roles

There are three application roles:

NORMAL_USER

Can:

- Register/login/logout
- Manage own profile
- Browse streams
- Watch streams
- Join stream chat
- Follow/unfollow creators
- Send gifts
- Use wallet credits
- Report users/content

Cannot:

- Create or manage streams
- Access admin APIs

CREATOR

Has all NORMAL_USER capabilities plus:

- Create streams
- Schedule streams
- Start own scheduled stream
- End own live stream
- Update own stream metadata
- View own stream analytics/basic viewer information
- Generate/use broadcaster media credentials

A creator is NOT an administrator.

ADMIN

Can:

- View users
- Disable/restore users
- View all streams
- End any active stream
- Review reports
- Moderate platform content
- Inspect operational information

Admin privileges must never be inferred from creator status.

3. Architecture Overview

[illegible]

■ Zylo Web UI ■

■ Next.js / React / TS ■

[REDACTED]

HTTPS REST + Socket.IO

▼

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

■ Node / Express API ■

■ Modular Monolith ■

[illegible]

■ ■

XX

▼ ▼

[REDACTED] [REDACTED]

■ PostgreSQL/Neon ■ ■ Redis ■

■ Source of truth ■ ■ Cache / Presence ■

■■ Rate limits ■■

■ Socket adapter ■

[illegible]

▼

[illegible]

MediaMTX

■ Free/self-hosted ■

■ RTMP/WebRTC/HLS ■

XXXXXXXXXXXXXXXXXXXX

[illegible]

▼ ▼ ▼

FFmpeg Live playback Recording

ABR/transcode WebRTC/HLS fMP4 segments

■ ■

[REDACTED]

▼

[illegible]

■ Cloudflare R2 ■

■ VOD recordings ■

■ HLS assets ■

[REDACTED]

- Media server can run locally or on one VPS/server.
- R2 stores recordings.
- Cloudflare CDN/caching can serve VOD assets.

- A future production deployment can replace the media provider with Cloudflare Stream, LiveKit, Mux, AWS IVS, etc.

Cloudflare Stream is not the free option for this MVP: its current model charges for stored and delivered video minutes. Therefore Cloudflare Stream should remain a future provider option, not the default MVP media backend.

5. Media Provider Abstraction

Do NOT couple business logic directly to MediaMTX.

Create:

```
interface MediaProvider {  
  createLiveInput(streamId: string): Promise;  
  getPublishCredentials(streamId: string): Promise;  
  getPlaybackInfo(streamId: string): Promise;  
  startRecording(streamId: string): Promise;  
  stopRecording(streamId: string): Promise;  
  terminateStream(streamId: string): Promise;  
}
```

Implementation:

```
MediaProvider  
■■■■ MediaMtxProvider
```

Future:

```
MediaProvider  
■■■■ MediaMtxProvider  
■■■■ CloudflareStreamProvider  
■■■■ LiveKitProvider  
■■■■ MuxProvider
```

The rest of the application must not care which media provider is being used.

6. Stream Lifecycle

6.1 Immediate live stream

Creator

↓

POST /api/v1/streams

↓

Backend validates creator ownership/role

↓

Create DB stream

↓

Create media path/input



Return publish credentials + playback info



Creator starts broadcasting



Media server detects publisher



Backend marks stream LIVE



Redis presence starts



Viewers join



Chat/gifts/presence active



Creator ends stream



Media recording finalized



Recording packaged



VOD uploaded to R2



DB recording status = READY



Stream status = ENDED

6.2 Scheduled stream

Creator



POST /api/v1/streams

status = SCHEDULED

scheduledAt = future timestamp



DB stores scheduled stream



Redis cache stores upcoming stream metadata



UI displays countdown/upcoming state



Creator starts stream



SCHEDULED → LIVE

↓

Normal live lifecycle

Do not automatically mark a scheduled stream LIVE merely because its scheduled time has arrived. The creator must actually start broadcasting.

A background job may mark stale scheduled streams as `EXPIRED` if desired, but the core required statuses remain:

SCHEDULED

LIVE

ENDED

7. Stream Data Model

streams

id

public_id

broadcaster_id

title

description

thumbnail_asset_id

status

scheduled_at

started_at

ended_at

viewer_count

peak_viewer_count

media_provider

media_path

recording_status

recording_id

created_at

updated_at

Recommended enums:

StreamStatus:

SCHEDULED

LIVE

ENDED

RecordingStatus:

NOT_STARTED

RECORDING

PROCESSING

READY

FAILED

DELETED

Indexes:

(status, scheduled_at)

(status, started_at)

(broadcaster_id, status)

(created_at)

Never use client-provided `broadcaster\_id`.

Always derive broadcaster identity from the authenticated user.

8. Scheduling API

Create stream

POST /api/v1/streams

Request:

```
{
  "title": "Late Night Session",
  "description": "Talking and music",
  "scheduledAt": "2026-10-10T18:30:00.000Z"
}
```

For immediate streaming:

```
{
  "title": "I'm Live",
  "description": "Live now"
}
```

Response:

```
{
  "success": true,
  "data": {
    "id": "stream_public_id",
    "status": "SCHEDULED",
    "scheduledAt": "2026-10-10T18:30:00.000Z"
  }
}
```

Upcoming streams

GET /api/v1/streams/upcoming

Support:

cursor

limit

creatorId

from

to

Update scheduled stream

PATCH /api/v1/streams/:id

Only the owning creator can modify a scheduled stream.

Start scheduled stream

POST /api/v1/streams/:id/start

Only the owning creator can start it.

9. Live Playback Architecture

Preferred playback order

Primary

WebRTC for low-latency live playback.

Fallback

HLS for browsers/network conditions where WebRTC is unavailable.

Viewer

↓

GET /streams/:id/playback

↓

Backend authorizes viewer

↓

Returns:

webRtcUrl

hlsUrl

availableQualities

liveLatencyMode

The backend does not proxy video bytes.

The browser connects directly to the media delivery endpoint.

This keeps Node.js out of the high-bandwidth media path.

10. Quality Selection

The application must support:

Auto
1080p
720p
480p
360p

Only expose qualities actually available for the stream.

Live

FFmpeg creates a quality ladder from the broadcaster input.

Example target ladder:

1080p ~ 4.5–6 Mbps
720p ~ 2.5–3.5 Mbps
480p ~ 1.0–1.8 Mbps
360p ~ 0.6–1.0 Mbps

Actual bitrate should be configurable.

Do not assume every input supports 1080p.

VOD

After recording:

source

↓

FFmpeg

■■■ 1080p

■■■ 720p

■■■ 480p

■■■ 360p

↓

HLS master playlist

↓

R2

Master playlist:

master.m3u8

Variant playlists:

1080p/index.m3u8

720p/index.m3u8

480p/index.m3u8

360p/index.m3u8

The frontend player uses the master playlist for adaptive bitrate playback.

11. Playback Speed

Playback speed applies to recorded streams/VOD, not normal live playback.

Supported:

0.5x

0.75x

1x

1.25x

1.5x

2x

The frontend player controls playback speed.

No backend request is required for changing playback speed.

Optional future feature:

PATCH /users/me/preferences

to remember the user's preferred playback speed.

12. Recording Pipeline

Recording flow

Broadcaster

↓

MediaMTX

↓

fMP4 recording segments

↓

Local temporary storage

↓

Upload worker

↓

R2

↓

FFmpeg packaging/transcoding

↓

R2 VOD hierarchy

Suggested R2 structure:

```
streams/  
{streamId}/  
source/  
recording.mp4  
hls/  
master.m3u8  
1080p/  
720p/  
480p/  
360p/  
thumbnails/  
poster.jpg
```

For the MVP, processing can happen after the stream ends.

Do not make the Node API server responsible for long-running video transcoding.

13. Cloudflare R2

Use Cloudflare R2 for recorded streams/VOD assets.

R2 is preferable here because it provides object storage with no Internet egress charge and currently includes a monthly free allowance for Standard storage and operations.

R2 is object storage, not the live media server.

Responsibilities:

R2:

- recordings
- HLS segments
- HLS playlists
- VOD thumbnails

Backend:

- authorization
- metadata
- signed/controlled access
- recording lifecycle

Frontend:

- playback

Never send R2 secret keys to the browser.

14. Recording Worker

Create a separate worker process/module:

backend/

src/

workers/

recording/

recording.worker.ts

recording.service.ts

recording.processor.ts

Responsibilities:

1. Detect recording completion
2. Mark recording `PROCESSING`
3. Validate media
4. Generate thumbnail
5. Generate VOD HLS ladder
6. Upload artifacts to R2
7. Verify uploaded objects
8. Update database
9. Mark recording `READY`
10. Retry failed jobs

Use a Redis-backed queue.

Recommended:

BullMQ + Redis

Jobs:

recording.process

recording.upload

recording.transcode

recording.cleanup

thumbnail.generate

Job id must be idempotent.

Example:

recording:{streamId}

A job retry must not create duplicate VOD records.

15. Redis Architecture

Redis is NOT the source of truth for permanent business data.

Use Redis for ephemeral/high-frequency workloads.

Redis responsibilities

1. Online presence

Keys:

presence:user:{userId}

presence:stream:{streamId}

Use TTL heartbeats.

Example:

presence:user:123 = online

TTL = 45 seconds

Heartbeat:

every 15 seconds

If heartbeats stop, the key expires.

This avoids expensive permanent presence writes to PostgreSQL.

16. Stream Viewer Presence

For every stream:

stream:{streamId}:viewers

Use a Redis Set for unique viewers:

SADD stream:{streamId}:viewers {userId}

SREM stream:{streamId}:viewers {userId}

SCARD stream:{streamId}:viewers

For connection-level tracking:

stream:{streamId}:connections

Map socket connection IDs to users.

Do not count multiple browser tabs as multiple users if product analytics require unique viewers.

Recommended:

user → one logical viewer membership

socket → connection tracking

17. Viewer Count Reconciliation

Redis gives fast counts.

PostgreSQL stores durable snapshots.

Flow:

Viewer joins

↓

Redis SADD

↓

SCARD

↓

Broadcast viewer count

↓

Periodic reconciliation

↓

PostgreSQL streams.viewer_count

Write durable viewer count periodically, not on every join/leave.

Recommended:

every 5–15 seconds

Also record:

peak_viewer_count

using atomic Redis operations.

18. Redis Cache Strategy

Cache live stream discovery

Key:

streams:live

TTL:

5–15 seconds

Invalidate/update when:

stream starts

stream ends

stream title changes

Cache public profiles

Key:

profile:{userId}

TTL:

1–5 minutes

Invalidate after:

profile update

follow/unfollow

account status change

Cache stream detail

Key:

stream:{streamId}

Short TTL.

Do not cache highly mutable viewer count as the canonical stream object.

Return viewer count from Redis when available.

19. Cache Rules

Never cache:

- Wallet balance as authoritative state
- Gift transaction results as authoritative state
- Authorization decisions
- Password/session secrets
- Admin permissions
- Private user data

Cache:

- Public profiles
- Live stream lists
- Upcoming streams
- Gift catalog
- Public stream metadata
- Non-sensitive discovery data

20. Redis Rate Limiting

Use Redis atomic counters/sliding windows.

Limits:

Login:

5 attempts / 15 minutes / IP + account

Register:

5 attempts / hour / IP

Chat:

5–10 messages / 10 seconds / user

Gift:

small burst limit / user / stream

Follow:

reasonable burst limit / user

General API:

per-user + per-IP limits

Return:

429 Too Many Requests

with:

```
{  
  "success": false,  
  "error": {  
    "code": "RATE_LIMITED",  
    "message": "Too many requests"  
  }  
}
```

21. Socket.IO Architecture

Socket.IO is used for application realtime events, not for transporting the actual video.

Video:

MediaMTX → WebRTC/HLS → Browser

Realtime:

Browser ↔ Socket.IO ↔ Node ↔ Redis

Required events:

stream:join

stream:leave

chat:send

chat:message
stream:viewer_count
stream:status
gift:sent
presence:update

Socket authentication

Client connects with access token.

Server:

1. Verifies token
2. Loads user identity
3. Checks account status
4. Attaches user to socket
5. Allows authorized events

Never trust:

userId
broadcasterId
senderId

from socket payloads.

22. Chat Security Flow

socket authenticated
↓
stream exists
↓
stream status = LIVE
↓
user is allowed to participate
↓
message validated
↓
rate limit checked
↓
message persisted
↓
message broadcast

Message limits:

max length: 500 characters

trim whitespace
reject empty messages
reject invalid payloads
rate limit

Persist:

chat_messages

Use Redis only for transient delivery/presence.

23. Virtual Gifts

Gift catalog lives in PostgreSQL.

Example:

Rose 10

Heart 25

Star 50

Diamond 100

Client sends:

```
{  
  "giftId": "rose",  
  "quantity": 1,  
  "idempotencyKey": "uuid"  
}
```

Client NEVER sends:

price

balance

receiverId

Backend derives them.

24. Gift Transaction

Use a PostgreSQL transaction.

BEGIN

↓

Lock sender wallet

↓

Validate gift

↓

Validate quantity

↓

Validate stream LIVE

↓

Resolve broadcaster

↓

Check balance

↓

Deduct credits

↓

Create gift transaction

↓

Commit

↓

Emit gift:sent

If any operation fails:

ROLLBACK

Idempotency:

UNIQUE(sender_id, idempotency_key)

This prevents duplicate gift charges from retries/double-clicks.

25. Database Model

Required entities:

users

profiles

follows

streams

stream_viewers

chat_messages

wallets

gifts

gift_transactions

reports

Recommended security/session entity:

refresh_tokens

Recommended media entity:

media_assets

stream_recordings

media_assets

id
owner_id
provider
provider_public_id
resource_type
asset_type
original_url
delivery_url
width
height
format
bytes
status
created_at
updated_at

stream_recordings

id
stream_id
storage_provider
storage_key
status
duration_seconds
size_bytes
poster_key
master_playlist_key
created_at
updated_at

26. User/Profile APIs

POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/logout
POST /api/v1/auth/refresh

GET /api/v1/users/me
GET /api/v1/users/:id
PATCH /api/v1/users/me

POST /api/v1/users/:id/follow

DELETE /api/v1/users/:id/follow

27. Stream APIs

POST /api/v1/streams

GET /api/v1/streams

GET /api/v1/streams/live

GET /api/v1/streams/upcoming

GET /api/v1/streams/:id

PATCH /api/v1/streams/:id

POST /api/v1/streams/:id/start

POST /api/v1/streams/:id/join

POST /api/v1/streams/:id/leave

POST /api/v1/streams/:id/end

GET /api/v1/streams/:id/playback

GET /api/v1/streams/:id/recording

28. Media APIs

POST /api/v1/media/cloudinary/signature

POST /api/v1/media/complete

DELETE /api/v1/media/:id

Cloudinary uploads should preferably be direct-from-browser using a server-generated signed upload configuration.

The backend should never proxy large image uploads unless there is a security or transformation reason.

29. Wallet/Gift APIs

GET /api/v1/wallet

GET /api/v1/gifts

POST /api/v1/streams/:id/gifts

GET /api/v1/users/:id/gifts

30. Reports/Admin APIs

POST /api/v1/reports

GET /api/v1/admin/users

PATCH /api/v1/admin/users/:id/status

GET /api/v1/admin/streams

POST /api/v1/admin/streams/:id/end

GET /api/v1/admin/reports

PATCH /api/v1/admin/reports/:id

Admin routes require:

authenticated + ADMIN

31. Authentication Architecture

Use:

short-lived JWT access token

+

rotating refresh token

Recommended:

Access token:

10–15 minutes

Refresh token:

7–30 days

Refresh token:

- HttpOnly
- Secure in production
- SameSite configured appropriately
- Rotated on refresh
- Reuse detection
- Revocation support

Passwords:

Argon2id preferred

Fallback:

bcrypt

Never store plaintext passwords.

32. Authorization Architecture

Create reusable middleware:

requireAuth()

```
requireRole("CREATOR")
requireRole("ADMIN")
requireStreamOwner()
requireActiveAccount()
```

Ownership rule:

CREATOR can modify own stream only.

ADMIN can modify any stream.

Example:

POST /streams/:id/end

if ADMIN:

allowed

else if CREATOR:

allowed only if stream.broadcasterId === req.user.id

else:

forbidden

33. Cloudinary Media Architecture

Cloudinary is the media system for:

- Profile avatars
- Cover images
- Stream thumbnails
- Gift icons
- Optional user-uploaded image content

Do NOT upload the original through Node when direct browser upload is possible.

Flow:

UI

↓

POST /api/v1/media/cloudinary/signature

↓

Backend creates short-lived signed parameters

↓

UI uploads directly to Cloudinary

↓

Cloudinary returns asset metadata

↓

UI sends asset ID/public ID to backend

↓

Backend stores media_assets

34. Image Upload Optimization

Use Cloudinary delivery transformations:

f_auto

q_auto

These automatically select efficient browser formats and quality.

For avatars:

square

c_fill

w_256

h_256

f_auto

q_auto

For profile/creator cards:

w_480

f_auto

q_auto

For larger profile banners:

w_1280

f_auto

q_auto

For stream thumbnails:

16:9

c_fill

w_640

f_auto

q_auto

Use responsive delivery/srcset rather than shipping one huge original to every device.

Store the original asset reference, but deliver transformed CDN URLs.

35. Upload Validation

Backend must validate:

mime type

file size

asset type

authenticated owner

upload intent

Recommended image restrictions:

JPEG

PNG

WebP

AVIF

Do not trust the browser-provided MIME type alone.

Validate Cloudinary result metadata after upload.

Optional future:

- moderation
- perceptual hashing
- NSFW detection
- duplicate detection

36. Frontend Integration Architecture

Next.js should never scatter fetch calls across components.

Use:

src/

lib/

api/

client.ts

auth.api.ts

users.api.ts

streams.api.ts

gifts.api.ts

wallet.api.ts

media.api.ts

reports.api.ts

admin.api.ts

socket/

socket.client.ts

socket.events.ts

media/

player.ts

Recommended data layer:

TanStack Query

Use query keys:

["streams", "live"]

["streams", "upcoming"]

["stream", streamId]

["profile", userId]

["wallet"]

["gifts"]

37. Frontend Auth Flow

Login UI

↓

POST /auth/login

↓

Backend returns access token/session

↓

Refresh mechanism established

↓

GET /users/me

↓

Hydrate current user

↓

Route guards

UI must never determine whether a user is an admin merely by hiding a button.

The backend remains authoritative.

38. Home Page Integration

Home calls:

GET /api/v1/streams/live

Render:

Creator

Title

Thumbnail

Viewer count

Live status

Use Redis-backed server-side viewer counts.

Cache discovery responses briefly.

When a stream ends:

Socket event

+

query invalidation

should remove it from the live list without requiring a full page refresh.

39. Upcoming/Scheduled Page

Call:

GET /api/v1/streams/upcoming

Display:

Creator

Title

Thumbnail

Scheduled date/time

Countdown

Follow/notify action if implemented

When the creator starts:

stream:status

causes UI to update from:

UPCOMING → LIVE

40. Stream Page Integration

Route:

/stream/[id]

Initial load:

GET /streams/:id

GET /streams/:id/playback

Then:

connect Socket.IO

join stream room

Player:

WebRTC first

HLS fallback

Controls:

Quality:

Auto

1080p

720p

480p

360p

Speed for VOD:

0.5x

0.75x

1x

1.25x

1.5x

2x

Live UI:

viewer count

chat

gifts

follow

report

VOD UI:

seek

quality

speed

fullscreen

volume

41. Stream Socket Contract

Client:

```
socket.emit("stream:join", {  
  streamId  
});
```

Server:

```
socket.emit("stream:joined", {  
  streamId,  
  viewerCount  
});
```

Chat:

```
socket.emit("chat:send", {  
  streamId,  
  message  
});
```

Server:

```
socket.to(room).emit("chat:message", {  
  id,  
  streamId,  
  user: {  
    id,  
    username,  
    displayName,  
    avatar  
  },  
  message,  
  createdAt  
});
```

Viewer count:

```
socket.to(room).emit("stream:viewer_count", {  
  streamId,  
  viewerCount  
});
```

Gift:

```
socket.to(room).emit("gift:sent", {  
  gift,  
  sender,  
  quantity  
});
```

Stream state:

```
socket.to(room).emit("stream:status", {  
  streamId,  
  status  
});
```

42. Media Playback Contract

Backend response:

```
{
  "streamId": "abc123",
  "status": "LIVE",
  "provider": "MEDIAMTX",
  "live": {
    "webRtcUrl": "https://media.example/live/abc123",
    "hlsUrl": "https://media.example/live/abc123/index.m3u8"
  },
  "recording": null,
  "qualities": [
    "auto",
    "1080p",
    "720p",
    "480p",
    "360p"
  ]
}
```

For ended stream:

```
{
  "streamId": "abc123",
  "status": "ENDED",
  "recording": {
    "status": "READY",
    "hlsUrl": "https://cdn.example/streams/abc123/hls/master.m3u8",
    "durationSeconds": 3820
  }
}
```

The frontend never constructs private storage URLs by itself.

43. CDN/Cache Strategy

Browser/CDN cache

Cache aggressively:

avatars

thumbnails

gift icons

VOD HLS segments

VOD posters

Do not cache:

wallet responses
private account data
admin data
chat API responses
presence APIs

Cache headers

Static immutable assets:

Cache-Control: public, max-age=31536000, immutable

HLS segments:

long-ish immutable cache

Playlists:

short/no-cache depending on live/VOD mode

Live manifests should not be treated like immutable assets.

44. Stream Recording Access

For public ended streams:

GET /api/v1/streams/:id/recording

Backend checks:

stream exists

recording READY

visibility/publication rules

user/account status

Then return playback metadata.

Do not send the video through Express.

For private/future restricted streams, return short-lived signed URLs or use a tokenized CDN strategy.

45. Background Jobs

Use Redis + BullMQ.

Workers:

recording-worker

media-worker

cleanup-worker

notification-worker (optional)

analytics-worker

Core jobs:

process-recording

transcode-recording

upload-recording

generate-thumbnail

cleanup-stream

reconcile-viewer-count

expire-scheduled-stream

Jobs must be retryable and idempotent.

46. Scheduled Jobs

Recommended jobs:

Every minute

Check scheduled streams that are stale/expired.

Every 5–15 seconds

Reconcile active viewer counts.

Every few minutes

Clean stale presence entries if TTL recovery is insufficient.

Periodically

Cleanup:

- abandoned stream sessions
- temporary media
- failed processing artifacts
- expired cache entries where necessary

Redis TTL should do most ephemeral cleanup automatically.

47. Backend Project Structure

backend/

src/

app/

app.ts

server.ts
routes.ts

config/
env.ts

common/
errors/
middleware/
validators/
constants/
logger/
pagination/
utils/

infrastructure/
database/
prisma.ts
redis/
redis.ts
keys.ts
queue/
bullmq.ts
cloudinary/
cloudinary.ts
storage/
r2.ts
media/
media-provider.ts
mediamtx.provider.ts

modules/
auth/
users/
profiles/
follows/
streams/
chat/
wallet/
gifts/
media/
recordings/
reports/
admin/

realtime/
socket.ts

auth.ts
rooms.ts
events.ts
presence.ts

workers/
recording/
media/
cleanup/
reconciliation/

prisma/
schema.prisma
migrations/
seed.ts

tests/
auth/
users/
streams/
chat/
gifts/
media/
admin/

docs/
openapi.yaml

scripts/
media/
seed/

.env.example
README.md
package.json

48. Layering Rules

Use:

Controller

↓

Service

↓

Repository

↓

Prisma

External integrations:

Service

↓

Provider interface

↓

Cloudinary / R2 / MediaMTX

Do not put:

- business logic in controllers
- Prisma calls throughout controllers
- Redis calls throughout UI-facing components
- media credentials in frontend code
- wallet arithmetic in frontend code

49. Environment Variables

The user will provide service credentials in ``.env``.

Required:

NODE_ENV=

PORT=

APP_URL=

FRONTEND_URL=

DATABASE_URL=

DIRECT_DATABASE_URL=

REDIS_URL=

JWT_ACCESS_SECRET=

JWT_REFRESH_SECRET=

CLOUDINARY_CLOUD_NAME=

CLOUDINARY_API_KEY=

CLOUDINARY_API_SECRET=

CLOUDINARY_UPLOAD_FOLDER=

R2_ACCOUNT_ID=

R2_ACCESS_KEY_ID=

R2_SECRET_ACCESS_KEY=

R2_BUCKET_NAME=

R2_PUBLIC_BASE_URL=

R2_ENDPOINT=

MEDIAMTX_BASE_URL=

MEDIAMTX_API_URL=

MEDIAMTX_API_SECRET=

MEDIA_PUBLIC_BASE_URL=

CORS_ORIGINS=

RATE_LIMIT_WINDOW_SECONDS=

RATE_LIMIT_MAX_REQUESTS=

Optional:

FFMPEG_PATH=

FFPROBE_PATH=

BULLMQ_QUEUE_PREFIX=

LOG_LEVEL=

SENTRY_DSN=

Never commit real `.env`.

50. No Docker

Docker is intentionally excluded from this implementation.

Local services:

PostgreSQL:

Neon

Redis:

managed Redis provider

MediaMTX:

native binary/process

FFmpeg:

native binary

Cloudinary:

managed

R2:

managed

The README must explain:

how to install MediaMTX

how to install FFmpeg

how to configure environment variables

how to run API

how to run workers

how to run Next.js

51. Security Requirements

Input validation

Use:

Zod

for:

- body
- params
- query
- Socket.IO payloads

HTTP security

Use:

Helmet

CORS

secure cookies

request size limits

Authorization

Every sensitive operation must verify:

authentication

role

resource ownership

account status

Secrets

Never:

send Cloudinary API secret to client

send R2 secret to client

send MediaMTX admin credentials to client

Logs

Never log:

password

JWT

refresh token

Cloudinary secret

R2 secret

wallet secrets

52. Request IDs

Every request gets:

X-Request-ID

If supplied by a trusted upstream, validate it.

Include request ID in:

logs

errors

internal service context

Response:

```
{
  "success": false,
  "error": {
    "code": "STREAM_NOT_FOUND",
    "message": "Stream not found",
    "requestId": "req_123"
  }
}
```

53. Error Contract

All API errors use:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "Human-readable message",
    "details": {},
    "requestId": "req_123"
  }
}
```

Success:

```
{
  "success": true,
```

```
"data": {}  
}
```

Do not expose stack traces in production.

54. Database Concurrency

Use PostgreSQL transactions for:

- wallet operations
- gift transactions
- stream lifecycle transitions
- follow counter updates where necessary

Wallet:

```
SELECT ... FOR UPDATE
```

or equivalent Prisma transaction/locking strategy.

Stream lifecycle must prevent:

two simultaneous starts

two simultaneous ends

end after another admin already ended stream

Use conditional updates or transaction checks.

55. Stream State Machine

Allowed transitions:

SCHEDULED → LIVE

SCHEDULED → ENDED

LIVE → ENDED

Disallow:

ENDED → LIVE

ENDED → SCHEDULED

LIVE → SCHEDULED

unless an explicit future feature implements restart semantics.

56. Media Security

Creator publishing credentials must be:

- unique per stream
- short-lived where possible
- revocable
- never exposed in logs
- never reused between creators

Viewer playback authorization should be separated from publisher credentials.

The browser should receive only the minimum information needed to play.

57. Cloudinary Security

Recommended flow:

authenticated user
↓
request upload signature
↓
backend verifies asset type/owner
↓
signed parameters
↓
direct upload

Upload folders:

zylo/users/{userId}/avatar
zylo/users/{userId}/profile
zylo/streams/{streamId}/thumbnail
zylo/gifts

Never allow arbitrary folder paths from the browser.

58. Observability

Implement:

structured logs
request IDs
health endpoint
readiness endpoint
media health check
Redis health check
database health check
queue health

Endpoints:

GET /health

GET /ready

Example:

```
{
  "status": "ok",
  "services": {
    "database": "up",
    "redis": "up",
    "media": "up",
    "queue": "up"
  }
}

---
```

59. Swagger/OpenAPI

Document:

Auth

Users

Profiles

Follows

Streams

Scheduling

Playback

Wallet

Gifts

Media

Reports

Admin

For each endpoint document:

authentication

roles

request

response

errors

pagination

example

Socket.IO events must be documented separately in the README/API docs.

60. Testing Requirements

Minimum meaningful tests:

Auth

- register
- duplicate email
- login
- invalid login
- protected endpoint

Authorization

- normal user blocked from creator stream creation
- creator blocked from admin
- creator cannot end another creator's stream
- admin can end any active stream

Streams

- create immediate stream
- create scheduled stream
- start scheduled stream
- invalid lifecycle transition
- join live stream
- end stream

Chat

- authenticated send
- unauthenticated send blocked
- invalid message
- rate limit

Gifts

- valid gift
- insufficient balance
- invalid gift
- duplicate idempotency key
- concurrent gift safety

Media

- Cloudinary signature requires auth

- unauthorized media ownership blocked
- recording state transition

Target:

25–40 meaningful tests

not just 10 superficial tests.

61. Frontend Route Scope

Keep:

/

/login

/register

/explore

/following

/notifications

/wallet

/gifts

/profile/[id]

/settings

/settings/profile

/settings/account

/settings/notifications

/settings/privacy

/stream/[id]

/go-live

/admin

/admin/users

/admin/streams

/admin/reports

Do NOT build:

/messages

/messages/[conversationId]

There is no 1-to-1 messaging system in this scope.

Do not add a friends system.

Use:

Following

Creators

People you follow

62. Go Live UI

For CREATOR:

Go Live

↓

Title

Description

Thumbnail

Schedule or Go Live Now

↓

Create stream

↓

Media permissions/setup

↓

Publish

For scheduled stream:

Date

Time

Timezone

Backend stores UTC.

Frontend renders local timezone.

63. Creator Broadcast Flow

POST /streams

↓

GET /streams/:id/playback

↓

get publish credentials

↓

browser/OBS publishes to MediaMTX

↓

media server becomes active

↓

backend confirms publisher

↓

stream status LIVE

↓

Socket stream:status

Never mark a stream LIVE simply because the creator clicked "Go Live."

The media layer must actually be publishing.

64. Viewer Join Flow

Viewer opens /stream/:id

↓

GET stream metadata

↓

GET playback metadata

↓

initialize WebRTC/HLS player

↓

POST /streams/:id/join

↓

Redis presence

↓

Socket stream:join

↓

chat enabled

↓

viewer count updated

When leaving:

Socket disconnect / stream:leave

↓

Redis SREM

↓

viewer count broadcast

Use disconnect cleanup as a safety mechanism.

65. Offline/Failure Handling

If media server fails:

stream remains LIVE only while media heartbeat/publisher is valid

If publisher disconnects temporarily:

grace period

Example:

10–30 seconds

After grace period:

stream → ENDED

This avoids instantly ending a stream because of a short network interruption.

66. Cache Invalidation Rules

Profile update

Invalidate:

profile:{userId}

Follow/unfollow

Invalidate:

profile:{targetId}

profile:{currentUserId}

Stream start

Invalidate:

streams:live

streams:upcoming

stream:{id}

Stream end

Invalidate:

streams:live

stream:{id}

Gift catalog

Long TTL.

Invalidate only on admin catalog changes.

67. Scaling Plan

100 users

Single:

Node API

Redis

Neon

MediaMTX

FFmpeg

is enough for the MVP.

10,000 users

Separate:

API instances

Socket.IO instances

Redis

worker processes

media server

Use Redis Socket.IO adapter.

Separate media from API.

100,000+ users

Move toward:

Load balancer

multiple API instances

Redis cluster

multiple Socket.IO nodes

dedicated workers

multiple media ingress nodes

regional media servers

CDN

managed streaming provider or distributed media layer

event/analytics pipeline

The Node API must never become the video bottleneck.

68. Important Architectural Rule

PostgreSQL = durable truth

Redis = ephemeral/high-speed state

R2 = durable video object storage

Cloudinary = optimized image/media asset delivery

MediaMTX = live media transport

FFmpeg = processing/transcoding

Socket.IO = application realtime

Node/Express = business/API layer

Next.js = UI

Each system has one clear responsibility.

69. Performance Rules

API

- pagination everywhere lists can grow
- select only required columns
- avoid N+1 Prisma queries
- indexes for all high-frequency filters
- Redis cache public discovery
- compression where appropriate
- request body limits

Database

- indexed foreign keys
- compound indexes
- unique constraints
- transactions for money-like operations
- cursor pagination for feeds

Media

- browser connects directly to media/CDN
- no Express proxying video
- ABR
- HLS segmentation
- WebRTC for low latency
- Cloudinary transformations
- CDN cache for VOD assets

Frontend

- lazy load heavy player
- responsive images
- avoid shipping original image sizes
- query caching
- skeleton loading
- optimistic UI only where server reconciliation is safe

70. Definition of Done

The backend is complete when:

- Auth works
- Roles work
- Creator ownership works
- Profiles work
- Follow works
- Immediate live streams work
- Scheduled streams work
- WebRTC/HLS playback works
- Viewer presence works
- Viewer counts update in realtime
- Chat works
- Chat rate limiting works
- Gifts work transactionally
- Wallet cannot go negative
- Duplicate gifts are protected
- Streams record
- Recordings reach R2
- VOD playback works
- Quality selection works
- VOD speed selection works
- Cloudinary image upload works
- Images are optimized
- Redis cache/presence works
- Admin works
- Reports work
- Swagger works
- Health endpoints work
- Tests pass
- No secrets are committed
- Frontend consumes real API/Socket contracts
- No 1-to-1 messaging is implemented
- No friends system is implemented
- No Docker dependency exists

71. Implementation Order

Phase 1 — Foundation

Express

TypeScript

config

error handling
logging
request IDs
Prisma
Neon
Redis
Swagger
health checks

Phase 2 — Identity

Auth
JWT
refresh rotation
roles
users
profiles
follow

Phase 3 — Streams

streams
ownership
lifecycle
scheduling
MediaMTX provider
publish credentials
playback metadata

Phase 4 — Realtime

Socket.IO
Redis adapter
presence
viewer counts
chat
rate limiting

Phase 5 — Economy

wallet
gifts
transactions
idempotency
concurrency safety

Phase 6 — Media

Cloudinary uploads

R2

recording

FFmpeg

VOD HLS

thumbnails

quality ladder

Phase 7 — Admin

reports

users

streams

moderation

Phase 8 — Frontend Integration

auth

home

explore

following

profile

go live

stream

wallet

gifts

admin

Phase 9 — Hardening

tests

security review

rate limits

cache review

DB indexes

failure handling

README

Swagger

architecture diagram

72. Final Technology Matrix

```
|---|---|
| Web | Next.js + React + TypeScript |
| UI | Tailwind CSS |
| Data fetching | TanStack Query |
| API | Node.js + Express + TypeScript |
| ORM | Prisma |
| Database | PostgreSQL / Neon |
| Cache | Redis |
| Queue | BullMQ + Redis |
| Realtime | Socket.IO |
| Live media | MediaMTX |
| Transcoding | FFmpeg |
| Live playback | WebRTC |
| Fallback playback | HLS |
| VOD storage | Cloudflare R2 |
| Image/media CDN | Cloudinary |
| Auth | JWT + rotating refresh tokens |
| Validation | Zod |
| API docs | Swagger/OpenAPI |
| Testing | Jest + Supertest |
| Deployment | Managed services + native media/worker processes |
| Containers | None for this phase |
```

73. Final Product Boundary

This implementation is intentionally broader than the original assessment's basic CRUD MVP, but still avoids unrelated social-platform features.

Included

- Authentication
- Roles
- Profiles
- Following
- Live streaming
- Scheduling
- Live presence
- Viewer counts
- Chat
- Gifts
- Wallet
- Reports
- Admin
- Recording
- R2 VOD
- Quality selection
- VOD speed control
- Redis caching
- Redis presence

- Cloudinary media optimization
- Socket.IO realtime
- API documentation
- Tests
- Security
- Frontend integration

Excluded

- 1-to-1 messaging
- Friends system
- Real-money payments
- Complex recommendation AI
- Multi-guest/co-hosting
- Full creator monetization/payouts
- Full moderation AI
- Massive-scale multi-region media infrastructure
- Docker in this implementation phase

These can be added later without rewriting the business layer because the media, storage and realtime integrations are provider/adaptor based.

74. Engineering Principle

The most important architectural decision is to keep the application separated into four planes:

CONTROL PLANE

Node + PostgreSQL

Auth, users, streams, scheduling, gifts, permissions

REALTIME PLANE

Redis + Socket.IO

Presence, chat, viewer counts, events

MEDIA PLANE

MediaMTX + FFmpeg

Live video, transcoding, recording

STORAGE/DELIVERY PLANE

R2 + Cloudinary

VOD, HLS assets, images, thumbnails

The frontend consumes the control and realtime planes while connecting directly to the media/storage delivery layer.

This prevents the Node.js API from becoming a media bottleneck and gives the project a clean migration path from a free MVP to a managed streaming provider later.